

AI Session 2026 #1

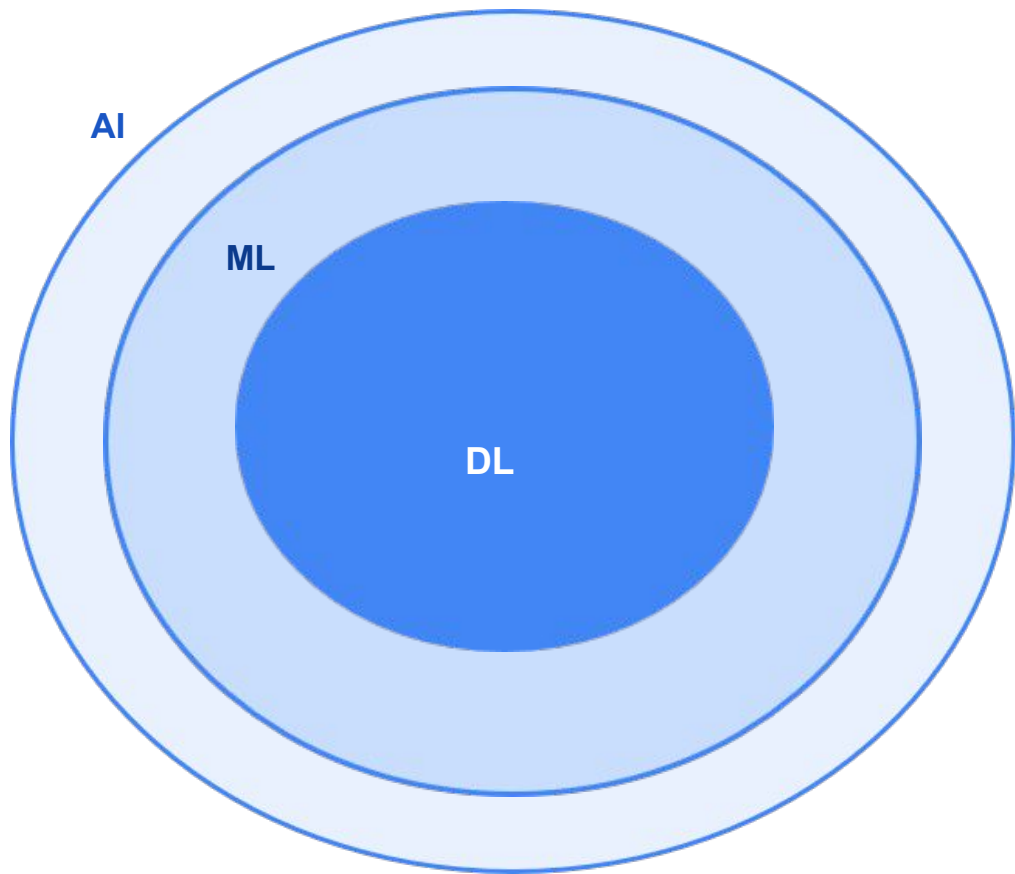
ML/DL

```
Lookup.KeyValue  
f.constant(['em  
=tf.constant([G  
lookup.StaticV  
_buckets=5)
```

머신러닝 · 딥러닝 기초

- 뉴런부터 시작하기

Intro - AI·ML·DL은 무슨 관계일까



인공지능 (AI)

사람처럼 사고·판단하도록 만드는 모든 기술 전체

머신러닝 (ML)

데이터에서 규칙을 스스로 찾아내는 AI의 한 방법

딥러닝 (DL)

뉴런을 층층이 쌓은 네트워크로 규칙을 찾는 ML의 한 갈래

TensorFlow Playground로 직접 보기

오늘 화면으로 직접 확인할
것

[TensorFlow Playground](#)

뉴런

은닉층의 사각형 노드 하나 = 계산 단위 하나

가중
치

노드 사이 선의 굵기, 색 = 학습되는 숫자

네트워
크

노드를 층으로 쌓은 전체 구조

학습

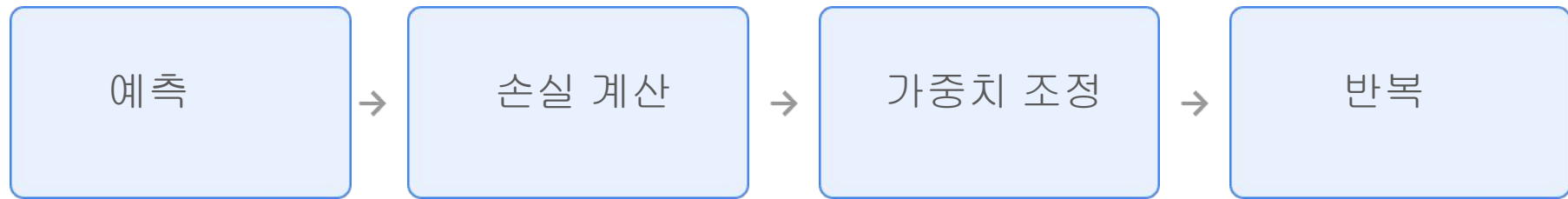
Loss 숫자가 줄고, 배경 경계선이 맞춰지는 과정

핵심 개념 정리 ① - 뉴런 / 가중치 / 네트워크

용어	정의	Playground에서 대응하는 것
뉴런	입력을 받아 숫자를 계산해 출력하는 단위	은닉층 사각형 노드 하나
가중치	그 계산에서 조절되는 숫자(학습 대상)	노드 사이 연결선
네트워크	뉴런을 여러 층으로 쌓은 구조	은닉층 전체

핵심 개념 정리 ② - 학습이란?

학습 = 손실을 줄이는 방향으로 가중치를 계속 조정하는 것



손실 (MSE): $L = (1/n) \sum (y - \hat{y})^2$ ← "얼마나 틀렸나"를 숫자 하나로

경사하강법: $w \leftarrow w - \eta \cdot \partial L / \partial w$ ← 그 숫자를 줄이는 방향으로 w 를 한 발 이동

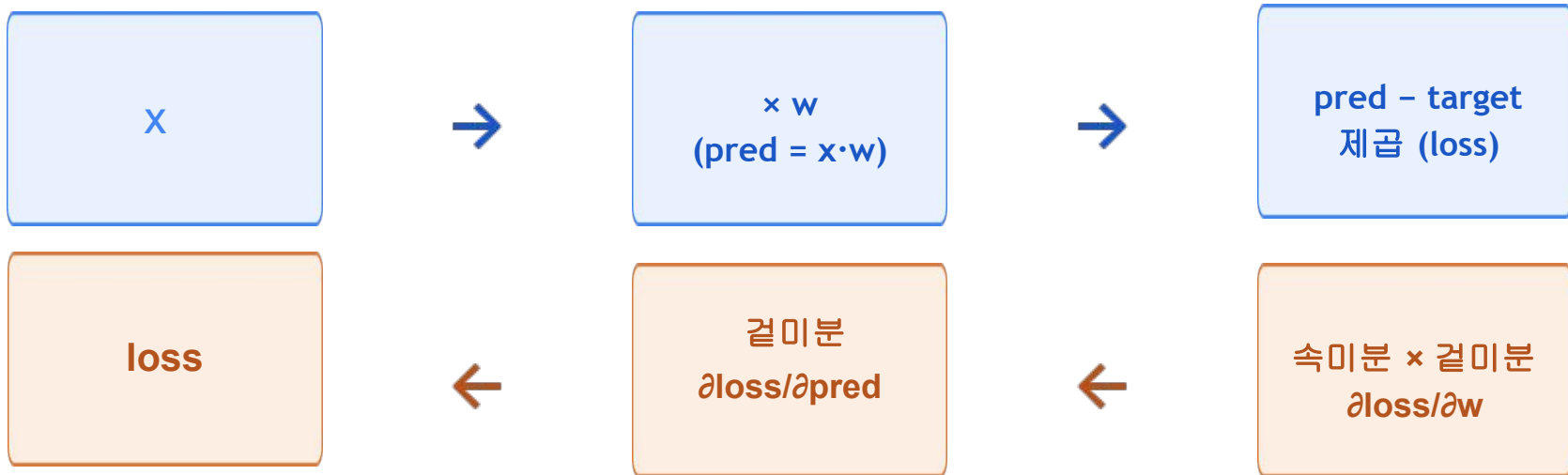
과적합 (Overfitting)

모델이 연습 문제(학습 데이터)만 통째로 외워서, 새로운 문제(실제 데이터)는 오히려 잘 못 푸는 상태

체인룰

순전파 (Forward) — 예측이 나오기까지, 앞으로 흘러가는 계산
줬는지

역전파 (Backward) — loss에서 거꾸로, 각 단계가 얼마나 영향을



체인룰: $\partial \text{loss} / \partial w = (\text{걸미분}) \times (\text{속미분}) = 2(\text{pred} - \text{target}) \cdot x$

→ `loss.backward()` 가 바로 이 곱셈을 자동으로 계산해준 것

간단 실습 - 손실 줄여보기

수식	코드
$L = (y - \hat{y})^2$	<code>loss = (pred - target) ** 2</code>
$\partial L / \partial w$ 계산	<code>loss.backward()</code>
$w \leftarrow w - \eta \cdot \partial L / \partial w$	<code>w -= 0.01 * w.grad</code>

```
import torch # torch 불러오기
x = torch.tensor([2.0])
w = torch.tensor([3.0], requires_grad=True)
target = torch.tensor([10.0])
pred = x * w
loss = (pred - target) ** 2
loss.backward()
with torch.no_grad():
    w -= 0.01 * w.grad
print(w.item()) # 정답(10) 쪽으로 살짝 이동
```

코드로 만나는 뉴런 - torch.nn

`nn` = neural network의 줄임말

PyTorch 안에 뉴런, 가중치를 자동으로 만들어주는 부품상자. 길게 쓰지 않으려고 줄여 부름.

```
import torch
import torch.nn as nn

# 방금 화면에서 본 뉴런 하나를 코드로
neuron = nn.Linear(1, 1, bias=False)

x = torch.tensor([2.0])
print(neuron(x))
```

뉴런에서 네트워크로

뉴런 하나로는 원형 데이터를 절대 못 가르친다. 이걸 쌓으면 `nn.Sequential` 한 줄이 됨.

```
network = nn.Sequential(  
    nn.Linear(2, 4),  
    nn.Linear(4, 2),  
    nn.Linear(2, 1)  
)
```

실습

오늘 써본 코드를 조금씩 바꿔보면서 복기

1

x 값을 2.0 → 5.0으로 바꿔서 실행 → 출력이 어떻게 달라지는지 확인

2

neuron을 다시 실행(재생성) → 값이 매번 달라지는지 확인 (무작위 초기화)

3

bias=False → bias=True로 바꿔서 실행 → 무엇이 추가로 나오는지 관찰

4

x를 `torch.tensor([2.0, 3.0, 4.0])`처럼 여러 개로 늘려서 넣어보기